Node.js — Day 20

Better Error Handling — Complete Notes

■ What We Learned

Today we learned how to detect, raise, catch, and handle errors in Node.js, and how this connects to HTTP API responses.

1. try/catch

try/catch lets us catch and handle an error when something fails. It does not prevent the error from happening.

```
try {
  const user = await getUser();
} catch (error) {
  console.log("Something went wrong:", error.message);
}
```

2. throw new Error()

throw is used when we detect a problem and want to raise an error.

```
throw new Error("User not found");
```

Example:

```
function getUser(id) {
  if (!id) {
    throw new Error("User ID is required");
  }
  return "User found";
}
```

3. async and Errors

An ordinary function can throw a synchronous error. An **async** function returns a Promise, so a thrown error becomes a rejected Promise and can be handled with **await** and **try/catch**.

Important: async is not required simply because try/catch is being used.

4. throw vs HTTP Response

throw raises the error. **catch** handles the error. The API's error-handling logic can then choose an appropriate HTTP status code.

throw → raises the error
catch → handles the error
HTTP status → tells the client what happened

5. API Error Handling

If a requested user does not exist, an API commonly responds with **404 Not Found**. An unexpected server-side failure may result in **500 Internal Server Error**.

Important: throw does not automatically send a 404. The API's error-handling code decides the HTTP response.

6. Node.js API Example

```
const http = require("http");

const server = http.createServer((req, res) => {
  try {
    const user = null;

    if (!user) {
      throw new Error("User not found");
    }
  } catch (error) {
    console.log(error.message);
    res.writeHead(404, {
      "Content-Type": "application/json"
    });
    res.end(JSON.stringify({
      error: "User not found"
    }));
  }
});

server.listen(3000);
```

7. Mini Challenge — Completed ■

You correctly implemented the missing-ID validation and **throw** logic. For the exercise, we used **return** instead of **res.end()** because the exercise was focused on error handling rather than HTTP responses.

■ Final Recap

• **try** → runs code that may fail.
• **catch** → handles an error that occurs.
• **throw** → intentionally raises an error.
• async is not required simply to use try/catch.
• An async function's thrown error becomes a rejected Promise.
• throw does not automatically create an HTTP status code.
• API error-handling logic chooses the appropriate HTTP response.
• Missing resource → commonly 404 Not Found.
• Unexpected server failure → commonly 500 Internal Server Error.

■ Progress

Day 20: COMPLETE ■
Next planned topic: Environment Variables ■